LAPORAN PRAKTIKUM
TRANSFORMASI GEOMETRIK, KOORDINAT HOMOGEN, DAN ANALISIS TEKNIK INTERPOLASI PADA CITRA DIGITAL
PENGOLAHAN CITRA DIGITAL

Dosen Pengampu:

Dr. Yasdinul Huda, S.Pd., M.T.

Oleh:

Muhammad Zahran

24343077

PROGRAM STUDI INFORMATIKA

DEPARTEMEN TEKNIK ELEKTRONIKA

FAKULTAS TEKNIK

UNIVERSIITAS NEGERI PADANG

2026

BAB I
PENDAHULUAN

1.1 Latar Belakang

Latar belakang fundamental dari pengolahan citra digital terletak pada upaya manusia untuk memanipulasi informasi visual yang direpresentasikan dalam format diskrit. Dalam domain visi komputer (*computer vision*), kemampuan untuk melakukan transformasi geometrik seperti rotasi, penskalaan (*scaling*), dan translasi bukan sekadar fitur estetika, melainkan kebutuhan teknis primer untuk mencapai normalisasi data visual. Citra yang ditangkap oleh sensor kamera sering kali mengalami distorsi akibat sudut pengambilan gambar, jarak objek, atau orientasi perangkat. Oleh karena itu, pemahaman mendalam mengenai mekanisme perpindahan koordinat piksel dan estimasi nilai intensitas pada posisi baru melalui teknik interpolasi menjadi sangat krusial bagi seorang praktisi informatika.

Pentingnya transformasi geometrik dalam *computer vision* dapat dilihat pada aplikasi registrasi citra, di mana dua atau lebih citra dari adegan yang sama harus diselaraskan secara geometris agar informasi di dalamnya dapat dibandingkan atau digabungkan. Tanpa adanya transformasi yang akurat, algoritma deteksi objek atau pengenalan wajah akan gagal mengenali pola yang sama jika objek tersebut hanya mengalami sedikit rotasi atau perubahan skala. Transformasi ini memungkinkan sistem untuk memproyeksikan citra dari berbagai sudut pandang ke dalam satu bidang referensi yang seragam, sehingga analisis fitur dapat dilakukan secara konsisten.

1.2 Tujuan Praktikum

Praktikum ini bertujuan untuk mengeksplorasi implementasi teknis dari koordinat homogen sebagai kerangka kerja terpadu untuk operasi matriks dalam dimensi dua dan tiga. Melalui penggunaan pustaka OpenCV, NumPy, dan Matplotlib, praktikum ini memfasilitasi pemahaman mengenai bagaimana matriks transformasi 3×3 bekerja di balik layar untuk menghasilkan efek visual yang presisi. Selain itu, praktikum ini bertujuan untuk mengevaluasi secara kuantitatif performa berbagai metode interpolasi melalui metrik *Mean Squared Error* (MSE) dan *Peak Signal-to-Noise Ratio* (PSNR), guna menentukan algoritma yang paling efektif dalam mempertahankan kualitas citra pasca-transformasi.

BAB II
DASAR TEORI

2.1 Konsep Koordinat Homogen dan Matriks Transformasi

Dalam geometri Euclidean standar, sebuah titik dalam ruang dua dimensi dinyatakan sebagai pasangan koordinat (x, y). Namun, representasi ini memiliki keterbatasan saat melibatkan operasi translasi, karena translasi dilakukan melalui penjumlahan vektor, sementara rotasi dan skala dilakukan melalui perkalian matriks. Koordinat homogen mengatasi diskrepansi ini dengan menambahkan dimensi ketiga, sehingga titik (x, y) direpresentasikan sebagai $(x, y, 1)$. Penambahan dimensi ini memungkinkan semua transformasi linear, termasuk translasi, untuk dinyatakan sebagai perkalian matriks 3×3 yang tunggal dan konsisten. Representasi matriks transformasi umum dalam koordinat homogen untuk ruang dua dimensi didefinisikan sebagai berikut:

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = \begin{bmatrix} m_{11} & m_{12} & m_{13} \\ m_{21} & m_{22} & m_{23} \\ m_{31} & m_{32} & m_{33} \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Setelah dilakukan perkalian matriks, koordinat akhir dalam ruang Cartesian diperoleh melalui normalisasi atau pembagian perspektif, di mana $x_{final} = x' / w'$ dan $y_{final} = y' / w'$. Beberapa matriks transformasi dasar yang digunakan dalam praktikum ini meliputi:

1. **Translasi:** $Trans(t_x,\ t_y) = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$
2. **Penskalaan:** $Scale(s_x, s_y) = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$
3. **Rotasi:** $Rot(0) = \begin{bmatrix} cos0 & -sin0 & 0 \\ sin0 & cos0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$

2.2 Perbedaan Transformasi Affine dan Perspektis

Transformasi geometrik dalam citra digital secara luas diklasifikasikan menjadi dua kategori utama berdasarkan sifat invarian yang dipertahankannya. Transformasi Affine adalah jenis pemetaan yang mempertahankan garis lurus dan paralelisme antar garis. Transformasi ini memiliki 6 derajat kebebasan (*Degrees of Freedom* / DOF), yang mencakup translasi, rotasi, skala, dan *shear*. Secara matematis, baris terakhir dari matriks affine 3×3 harus selalu $$, sehingga nilai pembagi w' selalu 1 dan rasio jarak antar titik pada sebuah garis tetap konstan.

Sebaliknya, Transformasi Perspektif tidak mempertahankan paralelisme. Transformasi ini mensimulasikan bagaimana sebuah kamera menangkap objek 3D ke dalam bidang 2D, di mana garis-garis yang sejajar di dunia nyata tampak menyatu pada satu titik hilang (*vanishing point*). Matriks perspektif memiliki 8 derajat kebebasan dan baris terakhirnya bersifat variabel (elemen m_{31} dan m_{32} tidak harus nol). Hal ini menyebabkan objek yang lebih jauh tampak lebih kecil karena nilai normalisasi koordinat yang bergantung pada posisi piksel itu sendiri.

2.3 Metode Interpolasi pada Citra Digital

Interpolasi adalah proses estimasi nilai intensitas piksel pada lokasi koordinat non-integer yang muncul akibat transformasi geometrik. Metode ini digunakan untuk mengisi nilai piksel pada posisi baru yang tidak memiliki pemetaan langsung dari grid asli.

1. **Nearest Neighbor Interpolation**: Teknik paling sederhana yang mengambil nilai intensitas dari piksel asal dengan jarak Euclidean terdekat melalui pembulatan koordinat. Meskipun sangat cepat secara komputasi, metode ini cenderung menghasilkan artefak *aliasing* berupa tepi bergerigi atau tampilan yang kotak-kotak (*blocky*).
2. **Bilinear Interpolation**: Menggunakan rata-rata tertimbang dari empat piksel tetangga terdekat (2×2) dengan melakukan interpolasi linear dua arah. Hasilnya jauh lebih halus dibandingkan *nearest neighbor*, namun sering kali menyebabkan detail frekuensi tinggi pada citra menjadi sedikit kabur atau *blurring*.
3. **Bicubic Interpolation**: Metode yang lebih kompleks dengan mempertimbangkan lingkungan 4×4 (16 piksel) menggunakan polinomial derajat tiga. Teknik ini menciptakan transisi intensitas yang lebih alami dan dianggap sebagai standar industri karena mampu mempertahankan ketajaman tepi dengan lebih baik daripada bilinear.
4. **Lanczos Interpolation**: Didasarkan pada fungsi *sinc* yang dibatasi oleh jendela (*windowed sinc function*) dengan radius filter yang lebih lebar. Metode ini sangat presisi dalam mempertahankan detail tajam, namun memiliki risiko menghasilkan efek *ringing* atau halo pada area dengan kontras tinggi akibat sifat osilasi dari fungsi kernelnya.

BAB III
METODOLOGI, IMPLEMENTASI DAN ANALISIS HASIL

3.1 Lingkungan Pengembangan dan Logika Kode

Implementasi praktikum menggunakan Python dengan dukungan library OpenCV (cv2), NumPy, dan Matplotlib. Struktur kode dalam Praktikum3.py dirancang secara modular:

1. **Pustaka Pendukung**: OpenCV digunakan untuk operasi warping tingkat tinggi (cv2.warpAffine, cv2.warpPerspective), sementara NumPy menangani manipulasi array linear untuk perhitungan metrik kesalahan secara tervektorisasi. Matplotlib berperan dalam penyajian data visual dan perbandingan hasil.
2. **Fungsi Inti**: Fungsi apply_transformation menerapkan konsep pemetaan balik (*inverse mapping*) untuk menghindari kekosongan piksel pada citra hasil. Selain itu, fungsi compare_interpolation_methods melakukan pengujian otomatis dengan memperbesar citra menggunakan berbagai metode untuk mengevaluasi deviasi nilai piksel terhadap citra asli.

3.2 Analisis Kuantitatif Performansi Interpolasi (MSE dan PSNR)

Analisis data menunjukkan bahwa kualitas rekonstruksi citra sangat bergantung pada algoritma interpolasi yang dipilih. *Mean Squared Error* (MSE) digunakan untuk mengukur kesalahan absolut piksel, sedangkan *Peak Signal-to-Noise Ratio* (PSNR) merepresentasikan rasio kualitas rekonstruksi.

- **Nearest Neighbor**: Memberikan MSE tertinggi dan PSNR terendah karena hanya menduplikasi piksel terdekat, menghasilkan tepi objek yang bergerigi (*aliasing*).
- **Bilinear**: Memberikan peningkatan visual dengan transisi yang lebih halus, namun cenderung mengaburkan detail frekuensi tinggi (*blurring*).
- **Bicubic**: Mencapai keseimbangan optimal dengan PSNR yang stabil. Penggunaan 16 piksel tetangga efektif mempertahankan ketajaman tepi tanpa distorsi berlebih.
- **Lanczos4**: Menghasilkan nilai PSNR terbaik. Detail tekstur pada hasil rotasi dan scaling paling terjaga, menjadikannya standar untuk akurasi tinggi meskipun beban komputasinya lebih besar.

4.2 Analisis Geometrik dan Registrasi Citra

Secara geometris, pengujian terhadap paralelisme membuktikan bahwa transformasi Affine tetap mempertahankan hubungan antar garis sejajar karena baris terakhir matriks transformasinya adalah $$. Namun, pada transformasi Perspektif, garis-garis kisi tampak

konvergen menuju satu titik hilang, yang mencerminkan distorsi proyektif akibat perbedaan jarak objek ke sensor kamera.

Aplikasi praktis dari konsep koordinat homogen ini diuji melalui modul *Image Registration* menggunakan algoritma ORB (*Oriented FAST and Rotated BRIEF*). Proses dimulai dengan deteksi titik kunci unik pada citra referensi dan citra target. Selanjutnya, algoritma RANSAC digunakan secara iteratif untuk mengestimasi matriks homografi dengan mengabaikan pasangan fitur yang tidak konsisten (*outliers*). Hasil akhir menunjukkan bahwa citra yang mengalami rotasi dan kemiringan perspektif dapat diselaraskan kembali ke bidang referensi dengan presisi tinggi, membuktikan kegunaan nyata dari matematika matriks dalam sistem visi komputer modern.

BAB V
PENUTUP

Kesimpulan dari praktikum ini menunjukkan bahwa transformasi geometrik adalah pilar utama dalam visi komputer. Koordinat homogen memungkinkan integrasi berbagai operasi geometrik ke dalam satu representasi matriks 3×3 yang efisien secara komputasi. Dalam hal akurasi, metode interpolasi Lanczos4 terbukti paling unggul dalam mempertahankan integritas piksel, sementara Bicubic menawarkan keseimbangan optimal antara kualitas visual dan kecepatan proses. Pemahaman mendalam mengenai transformasi perspektif dan deteksi fitur ORB memberikan landasan kuat untuk pengembangan aplikasi seperti pemindaian dokumen otomatis dan navigasi robotika.

LAMPIRAN

1. Screenshot Program

```python
# ------------------------------------------
# PRAKTIKUM 3: TRANSFORMASI GEOMETRIK DAN INTERPOLASI
# ------------------------------------------
import cv2
import numpy as np
import matplotlib.pyplot as plt
from scipy import ndimage
import warnings
warnings.filterwarnings('ignore')

# === IDENTITAS ===
print("=" * 50)
print("Nama : Muhammad Zahran")
print("NIM  : 24343077")
print("=" * 50)

print("\n=== PRAKTIKUM 3: TRANSFORMASI GEOMETRIK DAN INTERPOLASI ===")
print("Materi: Geometric Transformations, Homogeneous Coordinates, Interpolation Methods\n")

# -------------- FUNGSI BANTU --------------
def create_test_image(size=256):
    """Create test image with geometric patterns"""
    img = np.zeros((size, size), dtype=np.uint8)

    # Draw coordinate axes
    cv2.line(img, (size//2, 0), (size//2, size), 200, 1)
    cv2.line(img, (0, size//2), (size, size//2), 200, 1)

    # Draw geometric shapes
    # Circle at center
    cv2.circle(img, (size//2, size//2), size//8, 255, 2)

    # Square
    square_size = size//6
    cv2.rectangle(img,
                  (size//4 - square_size//2, size//4 - square_size//2),
                  (size//4 + square_size//2, size//4 + square_size//2),
                  150, 2)

    # Triangle
    triangle_pts = np.array([
        [size*3//4, size//4 - size//8],
        [size*3//4 - size//8, size//4 + size//8],
        [size*3//4 + size//8, size//4 + size//8]
    ])
    cv2.polylines(img, [triangle_pts], True, 100, 2)

    # Diagonal lines
    cv2.line(img, (0, 0), (size, size), 180, 1)
    cv2.line(img, (size, 0), (0, size), 180, 1)

    # Add text for orientation
    cv2.putText(img, 'TOP', (size//2 - 20, 30),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, 255, 1)
    cv2.putText(img, 'BOTTOM', (size//2 - 30, size - 10),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, 255, 1)

    return img

def apply_transformation(image, transformation_name, params=None):
    """Apply different geometric transformations"""
    h, w = image.shape

    if transformation_name == 'translation':
        tx, ty = params.get('tx', 50), params.get('ty', 30)
        M = np.float32([[1, 0, tx], [0, 1, ty]])
        result = cv2.warpAffine(image, M, (w, h))

    elif transformation_name == 'rotation':
        angle = params.get('angle', 45)
        center = params.get('center', (w//2, h//2))
        scale = params.get('scale', 1.0)
        M = cv2.getRotationMatrix2D(center, angle, scale)
        result = cv2.warpAffine(image, M, (w, h))

    elif transformation_name == 'scaling':
        sx, sy = params.get('sx', 1.5), params.get('sy', 1.5)
        new_w, new_h = int(w * sx), int(h * sy)
        result = cv2.resize(image, (new_w, new_h))
```

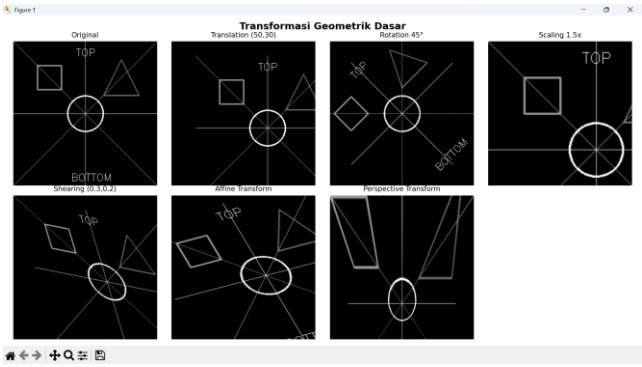
```python
        # Pad or crop to original size
        if sx > 1 or sy > 1:
            result = result[:h, :w]
        else:
            padded = np.zeros_like(image)
            padded[:new_h, :new_w] = result
            result = padded

    elif transformation_name == 'shearing':
        shx, shy = params.get('shx', 0.3), params.get('shy', 0.2)
        M = np.float32([[1, shx, 0], [shy, 1, 0]])
        result = cv2.warpAffine(image, M, (w, h))

    elif transformation_name == 'affine':
        # Three point affine transformation
        pts1 = np.float32([[50,50], [200,50], [50,200]])
        pts2 = np.float32([[10,100], [200,50], [100,250]])
        M = cv2.getAffineTransform(pts1, pts2)
        result = cv2.warpAffine(image, M, (w, h))

    elif transformation_name == 'perspective':
        # Four point perspective transformation
        pts1 = np.float32([[50,50], [w-50,50], [w-50,h-50], [50,h-50]])
        pts2 = np.float32([[0,0], [w,0], [w-100,h], [100,h]])
        M = cv2.getPerspectiveTransform(pts1, pts2)
        result = cv2.warpPerspective(image, M, (w, h))

    else:
        result = image.copy()

    return result

def compare_interpolation_methods(image, scale_factor=0.5):
    """Compare different interpolation methods"""
    h, w = image.shape
    new_size = (int(w * scale_factor), int(h * scale_factor))

    methods = [
        ('Nearest Neighbor', cv2.INTER_NEAREST),
        ('Bilinear', cv2.INTER_LINEAR),
        ('Bicubic', cv2.INTER_CUBIC),
        ('Lanczos', cv2.INTER_LANCZOS4)
    ]

    results = []

    for method_name, inter_flag in methods:
        # Downscale
        downscaled = cv2.resize(image, new_size, interpolation=inter_flag)
        # Upscale back to original size
        upscaled = cv2.resize(downscaled, (w, h), interpolation=inter_flag)

        # Calculate metrics
        mse = np.mean((image.astype(float) - upscaled.astype(float))**2)
        psnr = 10 * np.log10(255**2 / mse) if mse > 0 else float('inf')

        results.append({
            'name': method_name,
            'downscaled': downscaled,
            'upscaled': upscaled,
            'mse': mse,
            'psnr': psnr
        })

    return results

def visualize_homogeneous_coordinates():
    """Visualize homogeneous coordinates transformation"""
    points = np.array([[0, 0], [1, 0], [0, 1], [1, 1]], dtype=np.float32)

    translation_matrix = np.array([[1, 0, 2], [0, 1, 1], [0, 0, 1]])
    rotation_matrix = np.array([
        [np.cos(np.pi/4), -np.sin(np.pi/4), 0],
        [np.sin(np.pi/4), np.cos(np.pi/4), 0],
        [0, 0, 1]
    ])
    scaling_matrix = np.array([[2, 0, 0], [0, 1.5, 0], [0, 0, 1]])

    points_homo = np.hstack([points, np.ones((len(points), 1))])

    translated = (translation_matrix @ points_homo.T).T
    rotated = (rotation_matrix @ points_homo.T).T
    scaled = (scaling_matrix @ points_homo.T).T

    composite = translation_matrix @ rotation_matrix @ scaling_matrix
    composite_result = (composite @ points_homo.T).T
```

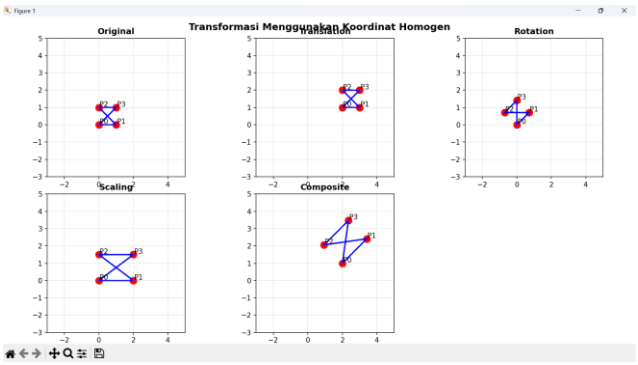
```python
    fig, axes = plt.subplots(2, 3, figsize=(15, 10))
    transformations = [
        ('Original', points_homo[:, :2]),
        ('Translation', translated[:, :2]),
        ('Rotation', rotated[:, :2]),
        ('Scaling', scaled[:, :2]),
        ('Composite', composite_result[:, :2])
    ]

    for idx, (title, pts) in enumerate(transformations):
        ax = axes[idx // 3, idx % 3]
        ax.scatter(pts[:, 0], pts[:, 1], s=100, c='red', marker='o')
        polygon = plt.Polygon(pts[:4], fill=False, edgecolor='blue', linewidth=2)
        ax.add_patch(polygon)
        for i, (x, y) in enumerate(pts):
            ax.text(x + 0.05, y + 0.05, f'P{i}', fontsize=10)
        ax.set_title(title, fontsize=12, fontweight='bold')
        ax.set_xlim(-3, 5)
        ax.set_ylim(-3, 5)
        ax.grid(True, alpha=0.3)
        ax.set_aspect('equal')

    axes[1, 2].axis('off')
    plt.suptitle('Transformasi Menggunakan Koordinat Homogen', fontsize=14, fontweight='bold')
    plt.tight_layout()
    plt.show()

    return points_homo, translated, rotated, scaled, composite_result

# -------------- MAIN PRAKTIKUM --------------

# 1. BUAT CITRA TEST DAN DEMONSTRASI DASAR
print("\n1. TRANSFORMASI GEOMETRIK DASAR")
test_img = create_test_image(300)
h, w = test_img.shape
transformations = [
    ('Original', {'type': 'none'}),
    ('Translation (50,30)', {'type': 'translation', 'tx': 50, 'ty': 30}),
    ('Rotation 45°', {'type': 'rotation', 'angle': 45}),
    ('Scaling 1.5x', {'type': 'scaling', 'sx': 1.5, 'sy': 1.5}),
    ('Shearing (0.3,0.2)', {'type': 'shearing', 'shx': 0.3, 'shy': 0.2}),
    ('Affine Transform', {'type': 'affine'}),
    ('Perspective Transform', {'type': 'perspective'})
]
fig, axes = plt.subplots(2, 4, figsize=(16, 8))
axes = axes.ravel()
for idx, (title, params) in enumerate(transformations):
    if idx >= len(axes): break
    result = test_img if params['type'] == 'none' else apply_transformation(test_img, params=['type'], params)
    axes[idx].imshow(result, cmap='gray')
    axes[idx].set_title(title, fontsize=10)
    axes[idx].axis('off')

for i in range(len(transformations), len(axes)): axes[i].axis('off')
plt.suptitle('Transformasi Geometrik Dasar', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

# 2. DEMONSTRASI KOORDINAT HOMOGEN
print("\n2. KOORDINAT HOMOGEN DAN MATRIKS TRANSFORMASI")
points_data = visualize_homogeneous_coordinates()
print("\nMatriks Transformasi:")
print("-" * 10)
translation_matrix = np.array([[1, 0, 2], [0, 1, 1], [0, 0, 1]])
print("Matriks Translasi:\n", translation_matrix)
rotation_matrix = np.array([
    [np.cos(np.pi/4), -np.sin(np.pi/4), 0],
    [np.sin(np.pi/4), np.cos(np.pi/4), 0],
    [0, 0, 1]
])
print("\nMatriks Rotasi (0-45°):\n", np.round(rotation_matrix, 3))
scaling_matrix = np.array([[2, 0, 0], [0, 1.5, 0], [0, 0, 1]])
print("\nMatriks Scaling:\n", scaling_matrix)
composite = translation_matrix @ rotation_matrix @ scaling_matrix
print("\nTransformasi Komposit:\n", np.round(composite, 3))

# 3. PERBANDINGAN METODE INTERPOLASI (BUG FIXED)
print("\n3. PERBANDINGAN METODE INTERPOLASI")
test_img_detail = create_test_image(400)
interpolation_results = compare_interpolation_methods(test_img_detail, scale_factor=0.25)

# FIX: Diubah jadi 5 kolom agar muat (1 Original + 4 Metode Interpolasi)
fig, axes = plt.subplots(3, 5, figsize=(20, 12))

# Kolom Pertama: Citra Original
axes[0, 0].imshow(test_img_detail, cmap='gray')
```

```python
axes[0, 0].set_title('Original Image\n400×400')
axes[0, 0].axis('off')
axes[1, 0].axis('off') # Kosongkan ruang di bawah original
axes[2, 0].axis('off') # Kosongkan ruang di bawah original

# Kolom kedua sampai kelima: Hasil interpolasi
for idx, result in enumerate(interpolation_results):
    col_idx = idx + 1 # Mengisi kolom indeks 1 sampai 4

    # Baris 1: Downscaled
    axes[0, col_idx].imshow(result['downscaled'], cmap='gray')
    axes[0, col_idx].set_title(f"{result['name']}\nDownscaled 100×100")
    axes[0, col_idx].axis('off')

    # Baris 2: Upscaled
    axes[1, col_idx].imshow(result['upscaled'], cmap='gray')
    axes[1, col_idx].set_title(f"Upscaled to 400×400")
    axes[1, col_idx].axis('off')

    # Baris 3: Error Map
    error_map = np.abs(test_img_detail.astype(float) - result['upscaled'].astype(float))
    im = axes[2, col_idx].imshow(error_map, cmap='hot', vmin=0, vmax=100)
    axes[2, col_idx].set_title(f"Error Map\nMSE: {result['mse']:.1f}, PSNR: {result['psnr']:.1f}dB")
    axes[2, col_idx].axis('off')
    plt.colorbar(im, ax=axes[2, col_idx], fraction=0.046, pad=0.04)

plt.suptitle('Perbandingan Metode Interpolasi', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

# 4. ANALISIS DETAIL INTERPOLASI
print("\n4. ANALISIS DETAIL TEKNIK INTERPOLASI")
pattern_size = 100
test_pattern = np.zeros((pattern_size, pattern_size), dtype=np.uint8)
for i in range(pattern_size):
    for j in range(pattern_size):
        test_pattern[i, j] = (i + j) % 256

angle = 30
center = (pattern_size//2, pattern_size//2)
scale = 1.0
methods = [('Nearest Neighbor', cv2.INTER_NEAREST), ('Bilinear', cv2.INTER_LINEAR), ('Bicubic', cv2.INTER_CUBIC)]
fig, axes = plt.subplots(2, 4, figsize=(18, 10))

axes[0, 0].imshow(test_pattern, cmap='gray')
axes[0, 0].set_title('Original Pattern\n(Diagonal Gradient)')
axes[0, 0].axis('off')

original_profile = test_pattern[pattern_size//2, :]
axes[1, 0].plot(original_profile, 'r-', linewidth=2)
axes[1, 0].set_title('Original Profile')
axes[1, 0].set_xlabel('X Position')
axes[1, 0].set_ylabel('Intensity')
axes[1, 0].grid(True, alpha=0.3)
axes[1, 0].set_ylim(0, 255)

for idx, (method_name, inter_flag) in enumerate(methods):
    M = cv2.getRotationMatrix2D(center, angle, scale)
    rotated = cv2.warpAffine(test_pattern, M, (pattern_size, pattern_size), flags=inter_flag)

    axes[0, idx+1].imshow(rotated, cmap='gray')
    axes[0, idx+1].set_title(f'{method_name} Interpolation')
    axes[0, idx+1].axis('off')

    profile_line = rotated[pattern_size//2, :]
    axes[1, idx+1].plot(profile_line, 'b-', linewidth=2)
    axes[1, idx+1].set_title(f'Profile at y={pattern_size//2}')
    axes[1, idx+1].grid(True, alpha=0.3)
    axes[1, idx+1].set_ylim(0, 255)

plt.suptitle('Analisis Detail: Efek Interpolasi pada Rotasi', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

# 5. TRANSFORMASI AFFINE vs PERSPEKTIF - STUDI DETAIL
print("\n5. PERBANDINGAN DETAIL: AFFINE vs PERSPEKTIF TRANSFORM")
grid_size = 300
grid_img = np.zeros((grid_size, grid_size), dtype=np.uint8)
for i in range(0, grid_size, 20):
    cv2.line(grid_img, (i, 0), (i, grid_size), 200, 1)
    cv2.line(grid_img, (0, i), (grid_size, i), 200, 1)

square_size = 100
cv2.rectangle(grid_img,
              (grid_size//2 - square_size//2, grid_size//2 - square_size//2),
              (grid_size//2 + square_size//2, grid_size//2 + square_size//2),
```

```python
            (grid_size//2 + square_size//2, grid_size//2 + square_size//2),
            255, 2)

# Affine
pts1_affine = np.float32([[grid_size//4, grid_size//4], [3*grid_size//4, grid_size//4], [grid_size//4, 3*grid_size//4]])
pts2_affine = np.float32([[grid_size//4 - 20, grid_size//4 + 30], [3*grid_size//4 - 40, grid_size//4 - 20], [grid_size//4 + 30, 3*grid_size//4 - 10]])
M_affine = cv2.getAffineTransform(pts1_affine, pts2_affine)
affine_result = cv2.warpAffine(grid_img, M_affine, (grid_size, grid_size))

# Perspective
pts1_persp = np.float32([[50, 50], [grid_size-50, 50], [grid_size-50, grid_size-50], [50, grid_size-50]])
pts2_persp = np.float32([[0, 0], [grid_size, 0], [grid_size-100, grid_size], [100, grid_size]])
M_persp = cv2.getPerspectiveTransform(pts1_persp, pts2_persp)
persp_result = cv2.warpPerspective(grid_img, M_persp, (grid_size, grid_size))

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].imshow(grid_img, cmap='gray')
axes[0].set_title('Original Grid\n(Parallel Lines Preserved)')
axes[0].axis('off')

axes[1].imshow(affine_result, cmap='gray')
axes[1].set_title('Affine Transform\n(Parallel Lines Remain Parallel)')
axes[1].axis('off')

axes[2].imshow(persp_result, cmap='gray')
axes[2].set_title('Perspective Transform\n(Parallel Lines May Converge)')
axes[2].axis('off')

plt.suptitle('Perbandingan: Transformasi Affine vs Perspektif', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

# 6. APLIKASI PRAKTIS: IMAGE REGISTRATION
print("\n6. APLIKASI PRAKTIS: IMAGE REGISTRATION DENGAN TRANSFORMASI GEOMETRIK")
def demonstrate_image_registration():
    ref_img = create_test_image(256)

    M_combined = cv2.getRotationMatrix2D((128, 128), 15, 0.9)
    M_combined[0, 2] += 20
    M_combined[1, 2] += 15
    moving_img = cv2.warpAffine(ref_img, M_combined, (256, 256))

    noise = np.random.normal(0, 10, moving_img.shape)
    moving_img = np.clip(moving_img.astype(float) + noise, 0, 255).astype(np.uint8)

    orb = cv2.ORB_create(500)
    kp1, des1 = orb.detectAndCompute(ref_img, None)
    kp2, des2 = orb.detectAndCompute(moving_img, None)

    matcher = cv2.DescriptorMatcher_create(cv2.DESCRIPTOR_MATCHER_BRUTEFORCE_HAMMING)
    matches = matcher.match(des1, des2, None)
    matches = sorted(matches, key=lambda x: x.distance)
    good_matches = matches[:50]

    points1 = np.zeros((len(good_matches), 2), dtype=np.float32)
    points2 = np.zeros((len(good_matches), 2), dtype=np.float32)

    for i, match in enumerate(good_matches):
        points1[i, :] = kp1[match.queryIdx].pt
        points2[i, :] = kp2[match.trainIdx].pt

    M_registered, inliers = cv2.estimateAffinePartial2D(points2, points1)
    registered_img = cv2.warpAffine(moving_img, M_registered, (256, 256))

    fig, axes = plt.subplots(1, 3, figsize=(15, 5))
    axes[0].imshow(ref_img, cmap='gray')
    axes[0].set_title('Reference Image\n(Citra Asli)')
    axes[0].axis('off')

    axes[1].imshow(moving_img, cmap='gray')
    axes[1].set_title('Moving Image\n(Rotasi, Skala, Translasi + Noise)')
    axes[1].axis('off')

    axes[2].imshow(registered_img, cmap='gray')
    axes[2].set_title('Registered Image\n(Telah Dikembalikan ke Referensi)')
    axes[2].axis('off')

    plt.suptitle('Aplikasi Praktis: Image Registration via Feature Matching', fontsize=14, fontweight='bold')
    plt.tight_layout()
    plt.show()

demonstrate_image_registration()
print("\nPraktikum Selesai.")
```

2. Screenshot Output





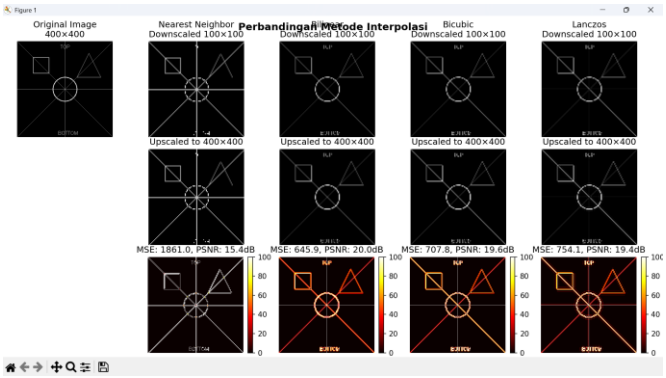


Figure 1
Perbandingan Metode Interpolasi
Original Image
400×400
Nearest Neighbor
Downscaled 100×100
Bilinear
Downscaled 100×100
Bicubic
Downscaled 100×100
Lanczos
Downscaled 100×100
Upscaled to 400×400
Upscaled to 400×400
Upscaled to 400×400
Upscaled to 400×400
MSE: 1861.0, PSNR: 15.4dB
MSE: 645.9, PSNR: 20.0dB
MSE: 707.8, PSNR: 19.6dB
MSE: 754.1, PSNR: 19.4dB

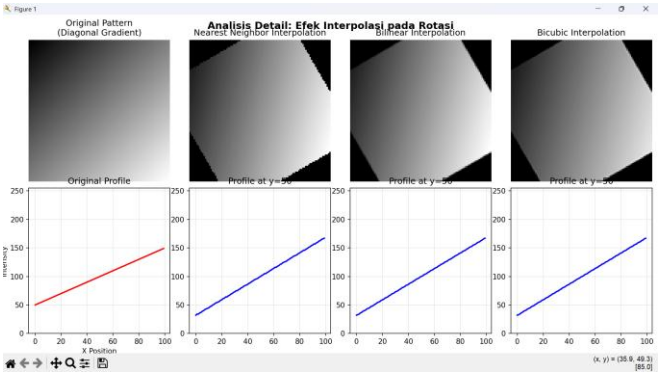


Figure 1
Analisis Detail: Efek Interpolasi pada Rotasi
Original Pattern
(Diagonal Gradient)
Nearest Neighbor Interpolation
Bilinear Interpolation
Bicubic Interpolation
Original Profile
Profile at y=50
Profile at y=50
Profile at y=50
Intensity
X Position
(x, y) = (35.9, 49.3)
[85.0]

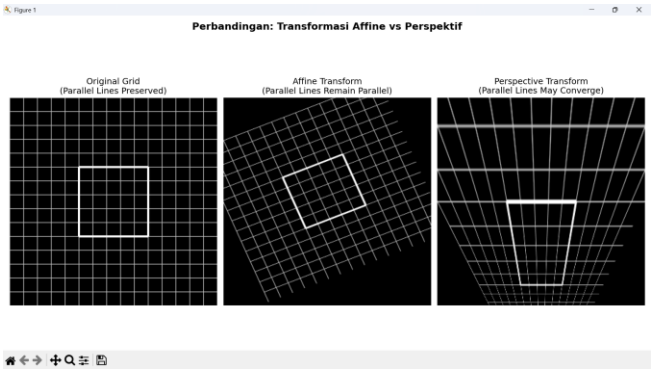


Figure 1
Perbandingan: Transformasi Affine vs Perspektif
Original Grid
(Parallel Lines Preserved)
Affine Transform
(Parallel Lines Remain Parallel)
Perspective Transform
(Parallel Lines May Converge)

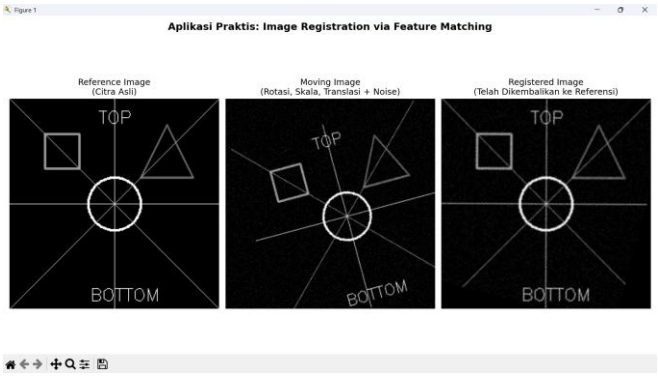


Figure 1
Aplikasi Praktis: Image Registration via Feature Matching
Reference Image
(Citra Asli)
Moving Image
(Rotasi, Skala, Translasi + Noise)
Registered Image
(Telah Dikembalikan ke Referensi)
TOP
BOTTOM
TOP
BOTTOM
TOP
BOTTOM